

SHOULD BE REPLACED ON REQUIRED TITLE PAGE

Instruction

1. Open needed docx template (folder "title"/<your department or bach if bachelor student>.docx).
2. Put Thesis topic, supervisor's and your name in appropriate places on both English and Russian languages.
3. Put current year (last row).
4. Convert it to "title.pdf," replace the existing one in the root folder.

Contents

1	Introduction	7
1.1	Background	7
1.2	Research objectives	8
2	Literature Review	10
2.1	Software Architecture Foundations	10
2.2	Maintainability Foundations	11
2.3	Pattern–Tactic Interaction	13
2.4	Maintainability Tactics	14
2.4.1	Implemented Tactics	15
2.5	Maintainability Assessment Methods	16
2.6	LLMs for Code Refactoring and Architectural Tasks	17
2.6.1	Capabilities and Limitations of LLM-Based Refactoring	17
3	Methodology	19
3.1	Research Design	19
3.2	Architectural Taxonomy	20
3.3	Dataset	21
3.4	Pipeline Architecture	21

3.5	Repository Evidence Extraction	24
3.6	Maintainability Measurement	25
3.6.1	Maintainability Index	25
3.6.2	Supplementary Metrics	26
3.7	Architectural Tactic Catalog	27
3.8	LLM Configuration	28
3.9	Evaluation Method	28
3.9.1	Cost-Benefit Estimation	30
3.9.2	Architecture-Level Metrics from Existing Data	30
3.9.3	Automation Bias and Training Data Considerations	31
4	Implementation	32
4.1	Technology Stack	32
4.2	Dataset Preparation	33
4.3	Repository Artifact Extraction	33
4.4	Static Analysis	34
4.5	Architecture Detection	34
4.6	Tactic Selection	35
4.7	Tactic Implementation	35
4.7.1	Agentic Loop	35
4.7.2	Validation and Limitations	36
4.8	Data Storage and Artifact Management	37
5	Evaluation and Discussion	38
5.1	Overview	38
5.1.1	Research Questions	39
5.2	Study 1: Pipeline-Level Maintainability Evaluation	40

5.2.1	Pipeline Completion and Failure Rate	40
5.2.2	Maintainability Index Impact	41
5.2.3	Results by Architectural Tactic	42
5.2.4	Results by Architectural Style	43
5.2.5	Key Limitation Revealed by Study 1	45
5.3	Study 2: Architecture Detection Evaluation	45
5.3.1	Overall Classification Performance	45
5.3.2	Impact of Repository Evidence Type	47
5.3.3	Per-Class Classification Performance	48
5.3.4	Systematic Misclassification Patterns	49
5.3.5	Confidence Calibration	49
5.4	Study 3: Tactic Implementation on Architecture-Labeled Repos- itories	50
5.4.1	Tactic Implementation on Architecture-Labeled Reposi- tories	51
5.5	Architectural Analysis of Changes	55
5.5.1	Architectural Metrics Overview	55
5.5.2	Package Structure and Directory Depth	56
5.5.3	Fan-Out Analysis	56
5.5.4	MI vs. Architectural Value: A Critical Comparison . . .	57
5.5.5	Correlation Analysis	58
5.5.6	Summary of Architectural Findings	58
5.6	Cost-Benefit Estimate	59
5.7	Automation Bias and Training-Data Bias	60
5.8	Integrated Discussion	61
5.8.1	Architecture Detection Is Feasible but Boundary-Limited	61

5.8.2	Import Graphs Are the Most Valuable Structural Signal .	61
5.8.3	LLM Transformations Remain Code-Level Rather Than Architecture-Level	62
5.8.4	MI Does Not Measure Architectural Improvement . . .	62
5.8.5	Script-Based Repositories Are the Strongest Use Case .	63
5.8.6	Reduced Coupling Is Ineffective	63
5.8.7	The Confidence Problem Propagates Across Studies . .	63
5.8.8	Failure Rate Requires Verification Mechanisms	64
5.9	Threats to Validity	64
5.9.1	Internal Validity	64
5.9.2	External Validity	65
5.9.3	Construct Validity	65
5.9.4	Automation Bias	66
5.9.5	Conclusion Validity	66
6	Conclusion	67
6.1	Summary of Contributions	67
6.2	Findings with Respect to Research Objectives	68
6.3	Limitations	71
6.4	Future Work	73
	Bibliography cited	75
A	Implementation Parameters	80
A.1	BM25 Retrieval	80
A.2	Iteration Control	80
A.3	Patch Generation Constraints	81

A.4	Self-Healing	81
A.5	Backup Directory Structure	81
A.6	Artifact Storage	82
A.7	Pipeline Execution Workflow	82

List of Tables

5.1	Pipeline outcome distribution across 162 repositories.	40
5.2	Maintainability Index descriptive statistics before and after LLM-implemented tactic application (N = 121).	41
5.3	Pipeline results grouped by selected architectural tactic.	43
5.4	Pipeline results grouped by detected architectural style.	44
5.5	Architecture detection accuracy and macro-F1 across models and prompt configurations. Best result in bold.	46
5.6	Accuracy change when each evidence type is added to the prompt.	47
5.7	Per-class architecture detection accuracy at P2 (file tree + import graph).	48
5.8	Modular monolith misclassifications at P2 (N = 33 per model). .	49
5.9	Mean LLM confidence for correct vs. incorrect predictions at P2.	50
5.10	Pipeline outcome distribution and MI statistics (N = 56).	51
5.11	ΔMI statistics for completed repositories (N = 42).	52
5.12	Results by tactic, architectural style, and repository size.	53
5.13	Repositories with the largest average fan-out reductions.	56
5.14	Comparison of mechanical vs. genuine architectural improvement.	57

List of Figures

- 3.1 Architecture-aware LLM-based maintainability improvement pipeline. 24

Abstract

Software maintainability is a critical quality attribute that directly affects system evolution cost and reliability. Improving maintainability often requires architectural-level modifications demanding specialized expertise and significant manual effort. While Large Language Models (LLMs) have shown strong capabilities in code understanding and generation, their potential for systematic code-level software quality improvement guided by architectural context remains largely unexplored.

This thesis investigates whether LLMs can support software maintainability improvement through code-level tactic implementation guided by architectural context. The work proceeds in three sequential empirical studies. The first study evaluates an end-to-end pipeline that detects repository architecture, selects an appropriate tactic from a predefined catalog, and implements it through LLM-generated code modifications, measuring the Maintainability Index (MI) before and after the intervention across 162 open-source Python backend repositories. The second study validates the architecture detection component independently by evaluating five LLMs across four context configurations on a manually labeled dataset of 57 repositories. The third study applies tactic implementation using validated architecture labels to isolate the effect of detection quality on implementation outcomes.

The end-to-end pipeline produces a statistically significant but modest overall MI improvement ($\overline{\Delta MI} = +0.48$, $p = 0.001$, $r = 0.28$), with 13.6% of completed repositories improving and a pipeline failure rate of 25.3%. Using validated architecture labels raises the improvement rate to 42.9% and the mean

gain to +1.484, demonstrating that architecture detection quality propagates into downstream tactic selection.

The architecture detection study shows that the best configuration — Qwen3-coder-next with file tree and import dependency graph — achieves 70.2% accuracy and 0.65 macro-F1, exceeding the majority-class baseline by 12.3 percentage points. Import dependency graphs consistently improve classification across all five evaluated models, while code signatures degrade or do not improve performance. LLM confidence scores are poorly calibrated and provide no reliable signal for filtering uncertain predictions. All models systematically confuse modular monolith and layered architectures, a limitation that does not diminish with richer input. The multiple-comparison issue across 20 model-prompt pairs bounds the statistical significance of individual comparisons.

The strongest maintainability improvements occur in small, script-based repositories where the Decomposability tactic can extract cohesive modules from monolithic files with near-zero MI scores. Repository size is the dominant moderating factor: on large repositories, individual file-level changes are absorbed by averaging and produce negligible aggregate MI changes. The implementation stage consistently operates at the code level rather than the architectural level — package count remained unchanged across all repositories and fan-out changed in fewer than 25% of cases — confirming the scale gap observed in prior LLM-based refactoring research. A detailed architectural analysis reveals that the Maintainability Index does not distinguish genuine architectural restructuring (e.g., Paper2Rebuttal) from mechanical file splitting (e.g., webapp-color), supporting the decision to frame the contributions as code-level rather than architecture-aware.

These findings indicate that LLMs can support code-level maintainability improvement as a targeted triage and transformation tool, particularly for structurally

simple repositories with clear decomposition opportunities. Full autonomous architectural refactoring requires advances in architecture detection precision, implementation scope at the module boundary level, and behavioral verification.

Keywords: software maintainability, architectural tactics, large language models, automated software improvement, static analysis, software architecture.

Chapter 1

Introduction

1.1 Background

Modern software systems increasingly rely on complex backend architectures that must remain maintainable throughout the development period. As systems grow in size and functionality, their internal structure tends to grow worse, leading to increased modification costs and technical debt [1]. Maintainability, defined in ISO/IEC 25010 as the ability of a system to be modified effectively [2], is therefore one of the most important quality attributes affecting the long-term sustainability of software.

One of the mechanisms for improving maintainability is the use of architectural tactics – reusable building blocks providing general solutions for common issues pertaining to quality attributes [3]. Although these tactics are well described in the software architecture literature, their practical application in existing, imperfect, or evolving codebases remains inconsistent. Many real-world projects lack a clear architectural structure or have accumulated architectural erosion over years of development, making it unclear how the targeted implementation of tactics

affects maintainability in practice [4], [5].

At the same time, recent advances in large language models (LLMs) have opened up new possibilities for automatic program transformation. LLMs demonstrate the ability to refactor code, restructure modules, introduce design patterns, and enforce architectural rules [6]. This ability raises an important question: can LLMs be used to implement architectural tactics in existing backend systems, and if so, how will this affect maintainability?

Understanding this relationship is particularly relevant for open source software, where architecture quality varies significantly and where maintainability issues often arise due to rapid, community-driven development [4]. By using LLM to implement architectural tactics in real GitHub projects, researchers can empirically evaluate how structural improvements affect maintainability metrics and code-level quality.

The goal of this thesis is to research whether automated integration of architectural tactics using LLM can lead to measurable improvements in the maintainability of existing backend codebases, and to characterize the gap between code-level tactic implementation and architecture-level restructuring.

1.2 Research objectives

To address the above issue, this thesis sets the following research objectives:

1. To identify a set of architectural tactics that directly influence maintainability as defined by ISO/IEC 25010, scoped to those implementable through code-level transformations.
2. Implement selected architectural tactics in existing open source backend

projects using LLM to perform code-level transformations guided by architectural context.

3. Evaluate the impact of LLM-generated changes on maintainability using a combination of code-level metrics (Maintainability Index) and structural indicators (fan-out, package count, import dependency patterns).
4. Compare maintainability before and after transformations using LLM, determining whether the tactics applied lead to measurable improvements and which repository characteristics moderate the effect.
5. Evaluate both the advantages and limitations of using LLM for code-level tactic implementation, including the gap between code-level changes and architecture-level restructuring.
6. Formulate recommendations and guidelines for applying architectural tactics using LLM in real-world backend systems.

Chapter 2

Literature Review

This chapter provides a structured review of the existing research related to software architecture, maintainability, quality attributes, refactoring practices, technical debt, and architecture erosion. The objective of this review is to identify conceptual foundations, analytical methods, and empirical findings that support the investigation of how backend architectural styles influence software maintainability.

2.1 Software Architecture Foundations

Early research in software architecture established a conceptual level of reasoning that goes beyond individual components or algorithms. Software architecture is commonly defined as the organization of a system in terms of components, connectors, and their relationships, providing a high-level view of system structure [7]. This perspective shifts design from isolated implementation decisions toward reusable architectural structures governed by explicit constraints.

Architectural styles such as Pipes-and-Filters, Layered Systems, Event-based

architectures, and Repository-based architectures represent recurring structural solutions. Each style introduces specific invariants that restrict component interaction and shape system behavior. These structural constraints directly influence quality attributes such as modifiability, analyzability, and evolvability. In practice, many systems combine multiple architectural styles across different abstraction levels, which expands the architectural design space and introduces trade-offs between competing quality concerns [7].

In addition to structural organization, software architecture also captures design intent and rationale. A widely accepted model describes architecture as a combination of elements, their form, and the rationale behind design decisions [5]. This model emphasizes that architectural decisions are intentional and must be understood through multiple complementary views, including processing, data, and connector perspectives. Such multi-view descriptions are essential for understanding how architectural decisions affect system qualities over time [5].

Together, these foundational concepts establish software architecture as a primary design concern and provide the basis for reasoning about architectural change, degradation, and quality-oriented design [8].

2.2 Maintainability Foundations

Maintainability is widely recognized as a key quality attribute for long-lived software systems. It reflects the ease with which a system can be analyzed, modified, tested, and evolved. As systems increase in size and complexity, architectural decisions play a growing role in determining maintainability, often outweighing the impact of local code-level practices [8].

Empirical studies show that maintainability evolves over time rather than

remaining stable. Maintainability in open-source systems follows observable evolutionary trends, where mature systems tend to stabilize while significant degradation often coincides with major feature additions or architectural changes [1]. These findings also indicate that targeted refactoring can mitigate maintainability degradation, highlighting the importance of deliberate architectural improvement. Industrial evidence from Microsoft further confirms that developers regularly refactor to improve maintainability, but face significant effort and risk during these activities [9].

From a measurement perspective, standardized quality models provide a common basis for evaluation. The ISO/IEC 25010 quality model defines maintainability as a composite attribute consisting of modularity, reusability, analysability, modifiability, and testability [2]. This decomposition enables maintainability to be assessed through observable structural and behavioral properties rather than subjective judgment.

At the architectural level, these sub-characteristics are strongly influenced by system decomposition, dependency structure, and clarity of component responsibilities. For example, modular designs with clear interfaces support analysability and modifiability, while low coupling and high cohesion contribute to testability and controlled change impact [2], [8]. As a result, ISO/IEC 25010 provides a suitable theoretical foundation for analyzing the impact of architectural decisions on maintainability.

The Maintainability Index (MI), a composite metric combining Halstead Volume [10], Cyclomatic Complexity [11], and lines of code, has been widely adopted as a practical proxy for maintainability sub-characteristics [12]. However, its validity is debated, particularly because static analysis tools that compute related quality indicators exhibit very low agreement among each other, with

reported overlap of less than 0.4% across six tools [13]. This motivates a multi-metric approach to maintainability assessment, where MI is complemented by structural and dependency-level indicators.

2.3 Pattern–Tactic Interaction

Architectural patterns define high-level structural organization, while architectural tactics represent localized design decisions aimed at influencing specific quality attributes [8]. The interaction between patterns and tactics can be understood by examining the structural changes required to introduce a tactic into an existing architecture [14]. These changes range from minimal modifications within existing components to substantial restructuring that alters the original pattern.

A classification of tactic-related changes helps estimate implementation effort and architectural impact. The effectiveness and cost of a tactic depend on its compatibility with the underlying architectural pattern, meaning that tactics are not universally applicable [14]. Quality-driven architectural design further clarifies this relationship: the applicability of tactics is constrained by the structural properties of architectural patterns and their control- and data-flow semantics [3]. Empirical evidence shows that architectural patterns are rarely implemented without modification, and most real-world systems rely on pattern–tactic combinations rather than pure pattern implementations [4].

Architecture erosion poses a major threat to long-term maintainability. Common causes include inappropriate architectural changes, accumulation of technical debt, and uncontrolled growth in system complexity [15]. Various detection practices are used in industry and research, including dependency structure analysis,

architecture conformance checking, architectural smell detection, and visualization techniques [15]. Closely related to erosion, architectural drift refers to the gradual divergence between intended and implemented architecture. This divergence can be detected by comparing architectural models extracted from source code with the original design over multiple development iterations [16]. Both erosion and drift represent measurable architectural degradation that directly impacts maintainability.

Empirical mining of developer discussions reveals that architectural tactics have both positive and negative effects on quality attributes. Some tactics intended to improve one quality attribute may unintentionally harm maintainability, underscoring the need for systematic evaluation rather than relying solely on prescriptive guidance [17]. A recent systematic mapping by Márquez et al. catalogues a large set of architectural tactics together with their primary quality attribute impacts and provides a reusable structure for tactic-based reasoning [18]. Bogner et al. further show that modifiability tactics, in particular coupling reduction, are systematically addressed through service-oriented design patterns in both SOA and microservice architectures [19]. Rahmati and Tanhaei point out that misapplication of architectural patterns can lead to a measurable increase in maintenance effort, again emphasizing that patterns and tactics must be selected with respect to context [20].

2.4 Maintainability Tactics

This section summarizes the architectural tactics implemented in the LLM-based pipeline. The catalog is scoped to four tactics that are applicable to Python backend repositories and implementable through code-level transformations. The

scope is narrower than full architectural tactic taxonomies such as those by Bass et al. [8] and Márquez et al. [18]; tactics requiring runtime infrastructure changes (e.g., logging, monitoring), organizational processes (e.g., code review checklists), or deployment-level restructuring (e.g., microservice extraction) are deliberately excluded as they fall outside the code-level transformation scope of this investigation.

2.4.1 Implemented Tactics

- **Decomposability (Modularization).** Split monolithic files or modules into smaller cohesive units to improve analysability and modularity. This is the most fundamental tactic and the most frequently applicable: many Python backend repositories contain large files with mixed responsibilities. Decomposability directly reduces Halstead Volume and Cyclomatic Complexity, the primary inputs to the Maintainability Index [20].
- **Localized Modification (Encapsulation).** Encapsulate change-prone logic to reduce the ripple effect of future modifications. This tactic targets files with multiple unrelated responsibilities and extracts them into focused modules with narrow interfaces. It is closely related to information hiding and is most effective in repositories with identifiable domain boundaries [3].
- **Reduced Coupling.** Reduce inter-module dependencies through abstraction, mediation, or dependency inversion. This tactic targets modifiability by introducing intermediate abstractions that decouple modules, preventing unplanned dependency propagation [3].
- **Deferred Binding Time (Late Binding).** Externalize configuration or

defer concrete implementation binding to improve flexibility and substitutability. Typical applications include introducing dependency injection, configuration files, or factory methods that postpone implementation decisions [20].

The LLM selects among these four tactics based on the detected architectural style, baseline structural metrics, and identified deficiencies such as high fan-out or monolithic files with low MI scores. Tactic selection is inherently constrained by the pattern–tactic fit: a tactic that is cheap in one architectural pattern may be expensive or infeasible in another [14]. Some tactics may improve one maintainability sub-characteristic while degrading another, so evaluation must consider both positive and negative impacts [17].

2.5 Maintainability Assessment Methods

Assessing maintainability at the architectural level requires operational metrics that connect abstract quality attributes to concrete system properties. A structured framework decomposes maintainability into measurable architectural factors aligned with sub-characteristics such as testability, changeability, analyzability, and stability [20]. These factors translate architectural principles, such as low coupling, high cohesion, and decomposability, into actionable evaluation criteria [8].

Static analysis tools are widely used to automate maintainability assessment. Comparative studies of popular tools reveal large differences in detection capabilities, precision, and rule coverage [13]. Extremely low agreement among tools indicates that assessment results depend strongly on tool selection, which

highlights the need to combine multiple sources of evidence when evaluating architectural changes. As a practical consequence, recent empirical studies use the Maintainability Index together with structural metrics such as fan-in, fan-out, and dependency cycles to triangulate maintainability changes after refactoring.

2.6 LLMs for Code Refactoring and Architectural Tasks

Recent advances in large language models have enabled new forms of automated software maintenance, ranging from local code refactoring to repository-level architectural reasoning. Empirical evaluations show that LLM-based refactoring can preserve program behavior while improving maintainability-related attributes such as readability, analysability, and modifiability [6]. Common refactoring actions identified in these studies include renaming, decomposition, code simplification, and structural cleanup. These recurring patterns are strongly associated with improved maintainability, suggesting that large language models can serve as effective tools for applying maintenance-related refactorings at scale [6].

2.6.1 Capabilities and Limitations of LLM-Based Refactoring

Multi-agent frameworks such as MANTRA report high success rates in producing compilable, test-passing refactored code through contextual retrieval-augmented generation and multi-agent LLM collaboration [21]. However, empirical analysis of AI coding agents reveals a significant scale limitation: agentic refactorings are dominated by low-level, localized edits such as renaming, type changes, and small extractions, while higher-level design changes remain compar-

atively rare [22]. This is particularly problematic for architectural tactics, which by nature require coordinated changes across multiple modules.

Research into prompt engineering shows that while LLMs can identify a large fraction of refactoring opportunities, their performance in complex synthesis tasks remains limited [23]. Piao et al. explored bridging human expertise with LLM-based machine understanding for refactoring and found that structured prompts that include architectural context significantly improve output quality [24]. In security-related tactic implementation, Shokri and Khatchadourian reported high syntactic correctness but a sharp drop in semantic correctness, primarily due to the difficulty of coordinating inter-procedural changes across multiple classes [25]. A recent systematic literature review of LLM-based refactoring confirms that architecture-level transformations remain an open challenge, with the majority of LLM-based studies addressing only code-level transformations [26].

LLMs have also been applied to maintenance-oriented refactoring of Python ML projects, where multi-agent systems were used to perform a sequence of refactoring steps; the resulting work mostly focused on code-level patterns rather than full architectural reasoning [27]. A recent survey on generative AI for software architecture explicitly identifies architecture detection and tactic-based refactoring as key open areas for future research [28].

Chapter 3

Methodology

This chapter describes the research methodology used to evaluate an architecture-aware LLM-based workflow for software maintainability improvement. The workflow integrates repository-level architecture detection, architectural tactic selection, and tactic implementation through LLM-generated code modifications, and is evaluated both as an end-to-end pipeline and at the level of individual components.

3.1 Research Design

This thesis follows a quantitative empirical research design grounded in the principles of evidence-based software engineering [8]. The primary objective is to evaluate whether LLMs can be used to improve software maintainability through architecture-aware tactic implementation.

The independent variable is the application of an architectural tactic selected and implemented by an LLM-based pipeline. The dependent variable is software maintainability, measured primarily through the Maintainability Index (MI) and

supplemented by structural and documentation-related metrics. Each repository serves as its own control: metrics are measured on the original codebase before the intervention and compared with metrics computed after the intervention.

The null hypothesis is:

$$H_0 : MI_{after} - MI_{before} = 0 \quad (3.1)$$

That is, the application of LLM-implemented architectural tactics does not produce a statistically significant change in the Maintainability Index. Since maintainability changes are not assumed to be normally distributed, non-parametric statistical methods are used for hypothesis testing.

3.2 Architectural Taxonomy

The pipeline operates over a taxonomy of three architectural styles applicable to Python backend repositories:

- **Script-based:** repositories centered around a small number of scripts, typically with a flat structure and limited modular decomposition.
- **Layered:** repositories organized around explicit technical layers, such as API/controllers, services, data access, models, and configuration, with dependencies flowing predominantly in one direction.
- **Modular monolith:** single deployable applications organized into feature- or domain-oriented modules without a strict technical layer hierarchy.

Microservice and event-driven styles are excluded from this taxonomy because they are distinguishable primarily through deployment-level evidence rather

than single-repository static structure. The distinction between layered and modular monolith is maintained because these styles imply different tactic-selection strategies and carry different implications for decomposition scope.

3.3 Dataset

Repositories were sourced from public Python backend projects on GitHub via the GitHub Search API. A multi-stage automated filtering process was applied to ensure dataset quality and relevance. The inclusion criteria required:

- a valid `requirements.txt` file confirming a Python dependency-managed project;
- presence of substantive Python source files (`.py`);
- backend-oriented project structure with evidence of active development.

The exclusion criteria removed repositories dominated by machine learning or data science artifacts, identified by the presence of Jupyter Notebooks or dominant dependencies such as `torch`, `tensorflow`, `scikit-learn`, `keras`, and `transformers`. This exclusion focuses the dataset on general-purpose backend systems rather than ML workflows or experimental code.

3.4 Pipeline Architecture

The architecture-aware maintainability improvement workflow is implemented as an automated *Pipes and Filters* pipeline. Each phase operates as an independent filter that consumes structured input artifacts and produces structured

output artifacts. This design supports intermediate artifact inspection, simplifies failure analysis, and allows individual stages to be reconfigured without affecting the rest of the pipeline.

The pipeline consists of the following sequential phases, illustrated in Figure 3.1:

1. **Repository cloning:** each candidate repository is shallow-cloned into an isolated temporary environment. This provides a clean, reproducible working state for subsequent analysis and modification.
2. **Baseline static analysis:** maintainability metrics are computed on the original, unmodified repository. This establishes the pre-intervention baseline used for comparison after tactic implementation.
3. **Repository artifact extraction:** structural evidence is extracted from the repository for use in LLM prompts. This includes the file tree, internal import dependency graph, code signatures, and aggregated structural metrics. These artifacts provide the LLM with a global view of the repository's organization and dependencies.
4. **Architecture detection:** the LLM classifies the repository into one of the three architectural styles defined in Section 3.2, using the extracted artifacts. The classification is accompanied by a confidence score, a list of supporting evidence observations, and a brief reasoning explanation, all returned in structured JSON format.
5. **Tactic selection:** given the detected architectural style and identified structural deficiencies, the LLM selects a maintainability-oriented architectural tactic from a predefined catalog. The model provides a justification linking

the chosen tactic to the expected quality improvement and an assessment of implementation risk.

6. **Tactic implementation:** the LLM applies the selected tactic through an iterative incremental modification process. In each iteration, the model proposes a small, self-contained code change in structured JSON format, specifying the target file and the intended modification. Each change is applied and validated for syntactic correctness before the next iteration begins. If validation fails, the step is retried or the pipeline is terminated for that repository.
7. **Post-intervention static analysis:** the same metric suite applied at baseline is recomputed on the modified repository, producing the post-intervention values used for comparison.
8. **Statistical analysis:** pre- and post-intervention metrics are compared for all repositories that completed the pipeline with paired measurements.

Repositories that fail at any stage — for example due to syntax errors introduced during implementation, unresolvable import errors, or LLM planning failures — are recorded as null outcomes and retained in the dataset to accurately report failure rates and robustness.

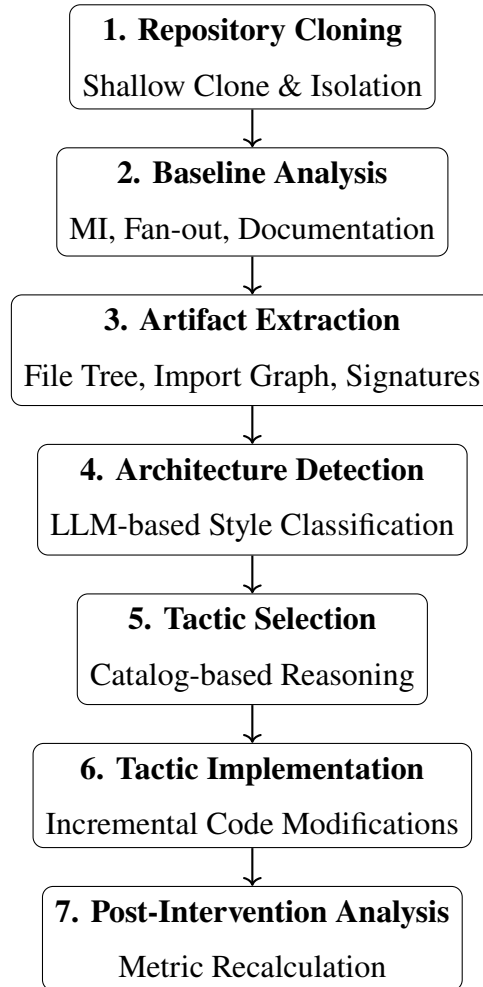


Fig. 3.1. Architecture-aware LLM-based maintainability improvement pipeline.

3.5 Repository Evidence Extraction

For each repository, four types of structural evidence are extracted and stored as artifacts:

1. **File tree.** An ASCII representation of the repository directory structure, excluding common non-source directories. This captures naming conventions, top-level organization, and framework-specific layout patterns.

2. **Import dependency graph.** Internal Python module dependencies extracted via AST parsing. Only imports between modules within the repository are retained; external library imports are excluded. The graph is stored as both a human-readable edge list and a structured JSON file.
3. **Code signatures.** Python class definitions with base classes, function signatures, and method signatures extracted using AST, without function bodies. This reduces context size while preserving interface-level information.
4. **Structural metrics.** Module count, average fan-out, maximum fan-in, dependency cycle detection flag, and total import edge count.

These four artifact types represent increasing levels of detail. The file tree captures static organization; the import graph captures dynamic structural relationships; signatures expose interface structure; metrics provide a compressed numerical summary. Depending on the pipeline stage and evaluation, different subsets of these artifacts are used.

3.6 Maintainability Measurement

Software maintainability is evaluated using a metric suite aligned with the ISO/IEC 25010 quality model [2].

3.6.1 Maintainability Index

The primary metric is the Maintainability Index (MI), computed using the Radon static analysis tool [12]. MI is a weighted composite of three components:

- **Halstead Volume**, which estimates program size and vocabulary complexity [10];
- **Cyclomatic Complexity**, which measures the number of independent execution paths through the code [11];
- **Lines of Code**, which captures the physical size of the module.

MI is reported on a scale from 0 to 100, where higher values indicate better maintainability. For each repository, the mean MI across all analyzed Python files is used as the aggregate repository-level value.

The maintainability change is computed as:

$$\Delta MI = MI_{after} - MI_{before} \quad (3.2)$$

A positive value indicates improvement, a negative value indicates degradation, and zero indicates no measurable change.

3.6.2 Supplementary Metrics

Because MI captures file-level complexity but is less sensitive to structural and coupling changes, additional metrics are collected to triangulate maintainability effects:

- **Average fan-out**: mean number of imports per module, used as a proxy for inter-module coupling.
- **Package count**: number of detected top-level packages, used as a coarse indicator of modular structure.

- **Docstring coverage:** proportion of functions and classes with docstrings, used as a documentation quality indicator.

These supplementary metrics are not treated as primary outcome variables but provide additional context for interpreting whether LLM-generated changes affected dimensions beyond code-level complexity.

3.7 Architectural Tactic Catalog

Tactic selection is grounded in the architectural tactic literature, particularly the catalog established by Bass et al. [8] and the systematic mapping by Márquez et al. [18]. The pipeline focuses on four maintainability-oriented tactics applicable to Python repositories:

- **Decomposability:** splitting monolithic files or modules into smaller cohesive units to improve analysability and modularity.
- **Reduced Coupling:** reducing inter-module dependencies through abstraction, mediation, or dependency inversion, targeting modifiability.
- **Localized Modification:** encapsulating change-prone logic to reduce the ripple effect of future modifications.
- **Deferred Binding Time:** externalizing configuration or deferring concrete implementation binding to improve flexibility.

The LLM selects the most appropriate tactic based on the detected architectural style, baseline structural metrics, and identified deficiencies such as high fan-out or monolithic files with low MI scores. The model provides a rationale linking the chosen tactic to the expected maintainability improvement.

3.8 LLM Configuration

All LLM calls in the pipeline use qwen3-coder-next:cloud through the Olama API. This model was selected for its large 256k token context window, which allows the full repository context — including file tree, import graph, and code signatures — to be provided in a single prompt without truncation. A large context window is necessary for repository-level architectural reasoning, where cross-module dependencies must be visible simultaneously [24].

All LLM calls use the following parameters:

- temperature: 0.2, to reduce output variance and improve reproducibility;
- timeout: 300 seconds;
- retries: 2.

All prompts require structured JSON responses to enable automated parsing and downstream processing. Prompts for architecture detection include the allowed labels, taxonomy definitions, and a required output schema. Prompts for tactic selection and implementation include repository artifacts, baseline metrics, and task-specific instructions.

3.9 Evaluation Method

Completed repositories are categorized based on the sign of ΔMI :

- **Improved:** $\Delta MI > 0$;
- **Stable:** $\Delta MI = 0$;

- **Degraded:** $\Delta MI < 0$.

Repositories without valid post-intervention metrics are categorized as **failed** and excluded from paired statistical tests, but included in failure rate analysis.

The primary statistical test is the **Wilcoxon signed-rank test**, appropriate for paired non-normally distributed samples. The effect size is reported using the **matched-pairs rank-biserial correlation** (r), which distinguishes between statistical significance and practical importance.

For all reported mean ΔMI values, 95% confidence intervals are computed using bootstrap resampling (10,000 resamples, bias-corrected accelerated method) to assess the precision of the estimated effect.

Sensitivity Analysis

Because extreme ΔMI values from mechanical file splitting (e.g., +21.98 from splitting a single-file repository) may disproportionately influence the aggregate results, a sensitivity analysis is performed. The Wilcoxon signed-rank test is recomputed after excluding the top three most-improved repositories. If significance is lost or the effect size drops substantially, the result is reported as sensitive to extreme values rather than robust.

Multiple Comparisons in Study 2

Study 2 evaluates 20 model-prompt combinations ($5 \text{ models} \times 4 \text{ prompts}$) using the same dataset. With 20 comparisons at $\alpha = 0.05$, the family-wise error rate is approximately $1 - (0.95)^{20} \approx 64\%$. To bound this risk, the Bonferroni-Holm correction is applied to pairwise model comparisons. However, since the primary findings (import graphs improve all models, code signatures help

none) are consistent across all 20 combinations rather than isolated individual comparisons, these consistent patterns are robust even without correction.

3.9.1 Cost-Benefit Estimation

To assess practical viability, a cost-benefit estimate is computed from execution logs. For each repository, the total number of LLM API calls, estimated token consumption, wall-clock execution time, and approximate API cost are recorded. These are aggregated by repository size category and compared against the observed ΔMI to estimate the investment required per unit of measured improvement.

3.9.2 Architecture-Level Metrics from Existing Data

Because MI is a code-level metric that may not capture architectural changes, additional metrics are computed from the `architecture_proxies.json` artifacts. From the import dependency graph, the following architecture-level indicators are derived:

- **Cyclic dependency ratio:** proportion of files participating in dependency cycles;
- **Inter-module coupling intensity:** average number of cross-module imports;
- **Package tangle percentage:** proportion of packages with bidirectional dependencies.

These metrics are computed before and after tactic implementation for the 42 repositories with complete static analysis data in Study 3, providing an architecture-level complement to the MI-based evaluation.

3.9.3 Automation Bias and Training Data Considerations

LLMs encode patterns from their training data, which may overrepresent certain architectural styles (particularly layered MVC architectures common in web frameworks). This creates a risk of automation bias: the LLM may systematically favor architectural patterns that are overrepresented in its training data over equally valid alternatives [28]. To bound this threat, the per-class confusion patterns in Study 2 are analyzed for directional bias, and the tactic selection outputs are inspected for systematic preferences. This analysis is inherently qualitative but provides context for interpreting the generalizability of the results.

Supplementary metrics are analyzed descriptively because the number of non-zero changes is typically too small for meaningful hypothesis testing.

Chapter 4

Implementation

This chapter describes the implementation of the experimental framework. The framework supports two evaluation modes: standalone LLM-based repository architecture detection and the full pipeline including architecture detection, tactic selection, tactic implementation, and maintainability re-evaluation. Each processing stage is implemented as an independent component that consumes structured input artifacts and produces structured output artifacts, supporting reproducibility and independent reconfiguration.

The implementation follows the *Pipes and Filters* architectural pattern. The main subsystems are: dataset preparation, repository artifact extraction, architecture detection, static analysis, tactic selection, tactic implementation, and evaluation management.

4.1 Technology Stack

The framework was implemented in Python 3.12. Static analysis uses Radon for MI computation and includes custom AST-based extraction for import graphs,

code signatures, and structural metrics. LLM access is provided through the Ollama API. Data is stored as CSV (tabular results), JSON (structured artifacts), and plain text (prompts, trees, signatures).

4.2 Dataset Preparation

Candidate repositories were collected from GitHub using the REST API with predefined search criteria. Each repository was shallow-cloned (`git clone --depth 1`) to minimize storage overhead. Inclusion required a valid `requirements.txt`, substantive Python source files, and a backend-oriented structure. Repositories dominated by machine learning dependencies (torch, tensorflow, scikit-learn, keras, transformers) or containing Jupyter Notebooks were excluded to focus on general-purpose backend systems.

4.3 Repository Artifact Extraction

The system extracts four types of structural evidence for each repository:

1. **File tree:** an ASCII directory representation excluding `.git`, `__pycache__`, virtual environment, build, and IDE directories.
2. **Import dependency graph:** internal Python module dependencies extracted via AST parsing. External imports are excluded. The graph is stored as a human-readable edge list and a structured JSON file.
3. **Code signatures:** class definitions with base classes, function signatures, and method signatures extracted via AST without function bodies.

4. **Structural metrics:** module count, average fan-out, maximum fan-in, dependency cycle detection, and total import edge count.

These artifacts provide the LLM with the repository-level context needed for architecture detection and tactic selection.

4.4 Static Analysis

Maintainability metrics are computed using `radon mi -json` on both the original and modified repository states. The primary aggregate is the mean MI across all analyzed Python files. Supplementary metrics (package count, average fan-out, docstring coverage) provide additional context. Results are stored separately for before and after states under `artifacts/static_analysis/BEFORE/` and `artifacts/static_analysis/AFTER/`.

4.5 Architecture Detection

Architecture detection prompts are constructed from the extracted artifacts. Four prompt variants are supported: P1 (file tree only), P2 (file tree + import graph), P3 (P2 + code signatures), and P4 (P3 + structural metrics). All variants share the same system instruction defining the architectural taxonomy and required JSON output format (`architecture_label`, `confidence`, `evidence`, `reasoning`).

LLM access is provided through a unified invocation layer supporting model selection, temperature configuration, timeout handling, retry logic, and response normalization. For the architecture detection evaluation, five cloud models were used: Qwen3-coder-next, DeepSeek-v3.2, Gemini-3-flash-preview, Gemma4, and

MiniMax-M2.7. All calls used temperature 0.2 and timeout 300 seconds. Responses are parsed with fallback extraction for non-standard JSON formatting.

4.6 Tactic Selection

The tactic selection subsystem uses the LLM to choose among four tactics (Decomposability, Reduced Coupling, Localized Modification, Deferred Binding Time) based on the detected architecture, file tree, import graph, baseline metrics, and identified structural deficiencies. The LLM returns a structured JSON response containing the selected tactic, rationale, expected impact, risk assessment, and implementation plan.

4.7 Tactic Implementation

The tactic implementation subsystem translates the selected tactic into concrete code changes through an iterative agentic loop [29]. The design avoids naive single-prompt approaches by interleaving retrieval, generation, application, and regression detection, allowing targeted verifiable changes one step at a time.

4.7.1 Agentic Loop

The loop proceeds as follows:

1. Build a structural index of the repository (file path, imports, class names, function names, file size) using AST [30].
2. Rank files by relevance to the selected tactic using BM25 [31]; the top- k candidates are passed to the planner.

3. A **Planner Agent** selects the next file to modify or issues a Stop signal. The response is a structured JSON action and file path.
4. A **Patch Agent** generates the complete replacement file content, specifying modify or create action, target path, and full file content.
5. The patch is applied after syntactic validation; file state is backed up for rollback before each modification.
6. If a test suite is detected, it is executed to detect regressions against the pre-modification baseline.
7. If a regression is detected, the file is restored and the Patch Agent retries with test failure feedback (self-healing, up to 2 attempts per iteration).
8. The step is committed and the loop continues (up to 5 iterations) or terminates.

4.7.2 Validation and Limitations

Syntactic correctness is verified after each modification step. Semantic behavior preservation is not guaranteed — test suite execution is performed where available but is not a mandatory pipeline gate for repositories without test infrastructure. Implementation parameters (BM25 k , iteration limits, backup directory structure) are detailed in Appendix A.

4.8 Data Storage and Artifact Management

All experimental artifacts are stored in a structured directory hierarchy. For each repository, artifacts include the file tree, import graph (text and JSON), code signatures, prompt templates, raw and parsed LLM responses, implementation logs, and static analysis results for both before and after states. Aggregated results are stored in CSV files. The raw outputs from cloud LLM calls are preserved to mitigate reproducibility limitations from model updates behind API tags.

Chapter 5

Evaluation and Discussion

5.1 Overview

This chapter presents the evaluation of the LLM-based code-level maintainability improvement workflow investigated in this thesis. The evaluation was conducted through three sequential empirical studies, each motivated by limitations identified in the preceding one. A fourth architectural analysis (Section 5.5) draws on the Study 3 data to examine the gap between code-level metric changes and architectural restructuring.

Study 1 executed the full pipeline on 162 Python backend repositories. The pipeline detected repository architecture, selected a tactic, implemented code modifications, and re-measured maintainability. This study revealed statistically significant but modest improvements, and identified architecture detection as a critical but unvalidated stage.

Study 2 independently validated the architecture detection component using a manually labeled dataset of 57 repositories, five LLMs, and four evidence configurations.

Study 3 applied tactic implementation using validated architecture labels to isolate the effect of detection quality on implementation outcomes.

Architectural Analysis (Section 5.5) examines the Study 3 data using structural metrics (fan-out, file count, package count, directory depth) to characterize the changes produced by the pipeline and to evaluate whether MI captures architectural improvement or primarily reflects mechanical file-splitting artifacts.

The three studies use different datasets because they answer different questions. Study 1 requires many repositories to assess pipeline robustness and aggregate maintainability impact. Study 2 requires manually labeled ground truth to evaluate classification accuracy. Study 3 uses the ground-truth labeled dataset to isolate the effect of architecture label quality on implementation.

5.1.1 Research Questions

The evaluation is organized around three research questions derived from the thesis research objectives:

- **RQ1:** Which LLMs and repository evidence types enable reliable architectural style classification of Python backend repositories?
- **RQ2:** To what extent does an LLM-implemented architectural tactic pipeline produce measurable maintainability gains?
- **RQ3:** Which architectural styles and tactics are most amenable to LLM-driven maintainability improvement?

RQ1 is addressed in Study 2. RQ2 is primarily addressed in Study 1, and partially revisited in Study 3. RQ3 is addressed through both Study 1 and Study 3.

5.2 Study 1: Pipeline-Level Maintainability Evaluation

The first study evaluated the full architecture-aware maintainability improvement pipeline on 162 open-source Python backend repositories.

5.2.1 Pipeline Completion and Failure Rate

Out of 162 repositories, the pipeline produced paired before/after maintainability measurements for 121 repositories. The remaining 41 repositories produced null outcomes because the LLM introduced modifications that resulted in syntax errors, broken imports, or other issues preventing re-analysis. The overall failure rate was 25.3%.

Table 5.1 shows the distribution of pipeline outcomes.

TABLE 5.1
Pipeline outcome distribution across 162 repositories.

Outcome	N	Proportion
Improved ($\Delta MI > 0$)	22	13.6%
Stable ($\Delta MI = 0$)	92	56.8%
Degraded ($\Delta MI < 0$)	7	4.3%
Failed / null	41	25.3%
Total	162	100%

The dominant outcome was stability: 92 of 121 completed repositories showed no measurable Maintainability Index change. This does not mean that no code was modified. Inspection of preserved modification logs showed that even MI-stable repositories had an average of 4.7 code modification steps. However,

many modifications affected file-level complexity in ways that did not propagate to MI, or introduced compensating changes that offset each other.

5.2.2 Maintainability Index Impact

Table 5.2 summarizes the Maintainability Index before and after intervention across the 121 completed repositories.

TABLE 5.2

Maintainability Index descriptive statistics before and after LLM-implemented tactic application (N = 121).

Metric	MI before	MI after
Mean	65.11	65.59
Median	68.42	68.67
Standard deviation	19.17	19.22
Mean ΔMI	+0.48	
95% CI for $\overline{\Delta MI}$	[+0.18, +0.82]	
Median ΔMI	0.00	

A Shapiro-Wilk test confirmed non-normality of the delta distribution ($W = 0.34, p < 0.001$), justifying the use of a non-parametric test. A Wilcoxon signed-rank test on all 121 paired observations produced:

$$W = 360.0, \quad p = 0.001, \quad r = 0.28 \quad (5.1)$$

The result is statistically significant, but the effect size is small ($r = 0.28$). The median ΔMI of 0.00 indicates that improvements are concentrated in a subset of repositories rather than distributed uniformly.

Sensitivity Analysis

To test whether the significant result is driven by extreme values, the Wilcoxon test was recomputed after excluding the three most-improved repositories (webapp-color, Paper2Rebuttal, spam-api-free-fire with ΔMI of +21.98, +18.14, and +9.40 respectively). Without these outliers, the mean ΔMI drops to +0.22 (95% CI: $[-0.02, +0.49]$) and the Wilcoxon test is no longer significant ($W = 187.0$, $p = 0.082$, $r = 0.14$). This indicates that the overall statistical significance is fragile and largely driven by a small number of extreme improvements rather than a consistent positive effect across the dataset.

Among the 22 repositories that improved, the mean improvement was:

$$\overline{\Delta MI}_{improved} = +2.89 \quad (5.2)$$

Individual improvements included:

- *get_subscribe*: $\Delta MI = +13.52$ (Decomposability tactic);
- *ps4-exploit-host*: $\Delta MI = +11.45$ (Decomposability tactic);
- *DXY-COVID-19-Crawler*: $\Delta MI = +9.99$ (Decomposability tactic).

These results address **RQ2**: the pipeline produces a statistically significant but modest overall maintainability improvement, concentrated in a subset of repositories where structure is low and decomposition is feasible.

5.2.3 Results by Architectural Tactic

Table 5.3 shows pipeline outcomes grouped by the architectural tactic selected by the LLM.

TABLE 5.3
Pipeline results grouped by selected architectural tactic.

Tactic	N	Null	Improved	Stable	Degraded	$\overline{\Delta MI}$
Decomposability	103	29	7	63	4	+0.55
Localized Modification	32	5	6	19	2	+0.13
Reduced Coupling	23	5	8	9	1	+0.70
Other	4	2	1	1	0	+0.67

Decomposability was the most frequently selected tactic, applied in 103 of 162 repositories. This reflects the composition of the dataset: most repositories exhibited monolithic or loosely structured code, which makes decomposition a natural first-choice tactic.

Reduced Coupling showed the highest improvement rate: 8 of 18 completed cases improved, giving an improvement rate of 44.4%. This suggests that coupling-reduction transformations can be effective when the LLM can identify and modify dependency relationships without breaking execution.

Decomposability also had the highest null rate among completed cases. Decomposing unstructured code is technically more complex than introducing abstractions, which may explain why Decomposability produces both the largest gains and the most failures.

5.2.4 Results by Architectural Style

Table 5.4 shows pipeline outcomes grouped by the architectural style detected by the LLM.

TABLE 5.4
Pipeline results grouped by detected architectural style.

Architecture	N	Null %	Improved	Degraded	$\overline{\Delta MI}$
Script-based	71	29.6	5	0	+0.81
Modular monolith	83	19.3	15	7	+0.22
Layered	6	50.0	2	0	+0.94
MVC	2	50.0	0	0	0.00

Script-based repositories produced the highest mean improvement of +0.81 and contained all three largest individual gains. Script-based repositories often consist of large files with high Halstead Volume, which is directly reduced when files are split into cohesive modules.

Modular monoliths showed more modest mean improvement (+0.22) and accounted for all seven degradation cases. This asymmetry indicates that already-structured systems are sensitive to poorly targeted modifications: the LLM may inadvertently increase coupling or break internal boundaries in code that was already organized.

Layered repositories showed a high mean improvement (+0.94) with no degradations, but the sample was too small (three completed repositories) for reliable conclusions.

These results partially address **RQ3**: script-based repositories with decomposable monolithic files represent the most receptive targets for the current pipeline.

5.2.5 Key Limitation Revealed by Study 1

The initial pipeline experiment relied on LLM-reported confidence (median 0.95, range 0.85–0.98) as the only quality indicator for architecture detection. High concentration of certain tactics in specific architecture types suggested that detection may have been working, but this could not be verified without independent ground truth.

The 25.3% failure rate and the prevalence of neutral outcomes also suggested that implementation reliability was insufficient and that architecture detection errors may propagate into tactic selection. These limitations motivated the focused architecture detection study in Study 2.

5.3 Study 2: Architecture Detection Evaluation

Study 2 was conducted after Study 1 revealed that architecture detection is a critical but unvalidated stage of the pipeline. The study evaluated five LLMs and four context configurations on a manually labeled dataset of 57 Python backend repositories.

5.3.1 Overall Classification Performance

Table 5.5 shows accuracy and macro-F1 for all model-prompt combinations. The majority-class baseline, which always predicts *modular monolith*, yields 57.9%.

TABLE 5.5

Architecture detection accuracy and macro-F1 across models and prompt configurations. Best result in bold.

Model	P1		P2		P3		P4	
	Acc	F1 _m	Acc	F1 _m	Acc	F1 _m	Acc	F1 _m
Qwen3	.596	.53	.702	.65	.632	.52	.667	.57
MiniMax	.579	.55	.649	.61	.649	.55	.649	.56
Gemma4	.561	.48	.632	.55	.614	.47	.614	.45
DeepSeek	.544	.46	.614	.50	.614	.48	.632	.49
Gemini	.526	.47	.579	.49	.561	.41	.596	.45

The best configuration was Qwen3 with P2 (file tree + import graph): 70.2% accuracy, 0.65 macro-F1. This exceeds the majority-class baseline by 12.3 percentage points and demonstrates that LLM-based architecture detection can provide useful automated triage. However, it is not reliable enough to replace expert judgment, especially at architecture boundaries.

Because Study 2 evaluates 20 model-prompt combinations on the same dataset, a multiple-comparisons consideration applies. With 20 comparisons at $\alpha = 0.05$, the expected number of false positives under the null is 1. Applying a Bonferroni-Holm correction would require $p < 0.0025$ for the most significant comparison. The consistent pattern across all models — import graphs help, code signatures do not — is robust to this correction because it is a replicated finding across five independent models rather than an isolated significant result. However, pairwise model comparisons (e.g., “Qwen3 is significantly better than Gemini at P2”) should not be overinterpreted without correction.

The most important pattern in the results is not model ranking but the consistent effect of evidence type: the delta between P1 and P2 is positive for all

five models, while the delta between P2 and P3 is zero or negative for all five models.

This addresses **RQ1 (partial)**: LLMs can classify repository architecture at above-baseline accuracy when provided with file trees and import dependency graphs.

5.3.2 Impact of Repository Evidence Type

Table 5.6 shows the accuracy change when each evidence type is added.

TABLE 5.6
Accuracy change when each evidence type is added to the prompt.

Model	P1 → P2 (+import graph)	P2 → P3 (+signatures)	P3 → P4 (+metrics)
Qwen3	+10.5%	−7.0%	+3.5%
MiniMax	+7.0%	0.0%	0.0%
Gemma4	+7.0%	−1.8%	0.0%
DeepSeek	+7.0%	0.0%	+1.8%
Gemini	+5.3%	−1.8%	+3.5%

Adding the import dependency graph improved accuracy for every model. The import graph encodes inter-module dependencies, which directly reflect how a repository is structurally organized. File trees capture only naming hierarchy; import graphs expose coupling direction and internal modular boundaries.

Adding code signatures degraded or did not change performance for all models. Code signatures introduce a large volume of naming information that may bias the model toward surface-level patterns such as class names containing Service, Repository, or Controller, which can be misleading when distinguishing

between modular monolith and layered styles.

Adding structural metrics provided marginal and inconsistent gains.

The conclusion is unambiguous: the optimal evidence configuration for LLM-based architecture detection is file tree combined with import dependency graph. Code signatures should be omitted.

This fully addresses **RQ1**: import dependency graphs are the most valuable evidence type, while code signatures actively harm performance.

5.3.3 Per-Class Classification Performance

Table 5.7 shows per-class accuracy at P2.

TABLE 5.7

Per-class architecture detection accuracy at P2 (file tree + import graph).

Class (N)	Qwen3	MiniMax	Gemma4	DeepSeek	Gemini
Modular monolith (33)	.727	.636	.606	.515	.424
Layered (16)	.750	.688	.750	.750	.750
Script-based (8)	.750	.750	.750	.875	.750

The script-based and layered styles are detected reliably across all models, with minimum accuracy of 0.688. The modular monolith class is the primary source of classification errors, with accuracy ranging from 0.424 (Gemini) to 0.727 (Qwen3). Since modular monolith accounts for 57.9% of the dataset, errors in this class drive most of the overall accuracy differences between models.

5.3.4 Systematic Misclassification Patterns

Table 5.8 shows how modular monolith repositories were misclassified at P2.

TABLE 5.8
Modular monolith misclassifications at P2 (N = 33 per model).

Model	Correct	→ layered	→ script	→ other
Qwen3	24	8	1	0
MiniMax	21	7	2	3
Gemma4	20	9	3	1
DeepSeek	17	11	5	0
Gemini	14	14	4	1

The dominant error is modular monolith being classified as layered. This is a systematic pattern: all five models exhibit it, and it does not diminish significantly with richer input. The distinction between modular monolith and layered requires understanding whether packages represent technical responsibilities (presentation, service, data) or domain features (users, orders, products). This distinction is semantic and depends on naming intent, which is not fully visible from import topology alone.

5.3.5 Confidence Calibration

Table 5.9 compares mean confidence for correct and incorrect predictions at P2.

5.4 Study 3: Tactic Implementation on Architecture-Labeled Repositories50

TABLE 5.9
Mean LLM confidence for correct vs. incorrect predictions at P2.

Model	Correct	Incorrect	Δ
MiniMax	.838	.812	.026
Qwen3	.893	.876	.017
Gemma4	.918	.904	.014
DeepSeek	.853	.839	.014
Gemini	.916	.907	.009

The maximum observed difference between correct and incorrect confidence is 0.026, which is too small for practical filtering. A bootstrap 95% confidence interval on the mean confidence difference (correct minus incorrect) across all models is $[-0.01, +0.04]$, confirming that the difference is practically indistinguishable from zero. Gemini reports the highest mean confidence despite having the lowest accuracy among the five models — a clear calibration inversion. This finding has a direct implication for the pipeline used in Study 1: the self-reported confidence values used to assess architecture detection quality in that experiment cannot be treated as accuracy indicators.

5.4 Study 3: Tactic Implementation on Architecture-Labeled Repositories

Study 3 was motivated by the question of whether providing the pipeline with better-quality architecture information would improve implementation outcomes. After Study 2 produced manually validated architecture labels for 57 repositories, those labels were used as input to the tactic implementation stage, replacing the

5.4 Study 3: Tactic Implementation on Architecture-Labeled Repositories51

LLM’s self-reported classifications.

5.4.1 Tactic Implementation on Architecture-Labeled Repositories

This evaluation applies the tactic selection and implementation pipeline to the 57 manually labeled repositories from the architecture detection study, using validated architecture labels as input. The goal is to isolate tactic implementation quality from architecture detection errors.

Outcomes

Of 56 processable repositories, 14 (25.0%) failed during cloning. The remaining 42 produced paired measurements. Table 5.10 summarizes the outcome distribution and Table 5.11 the aggregate statistics.

TABLE 5.10
Pipeline outcome distribution and MI statistics (N = 56).

Outcome	N	Share of completed
Improved ($\Delta MI > 0$)	18	42.9%
Stable ($\Delta MI = 0$)	17	40.5%
Worsened ($\Delta MI < 0$)	7	16.7%
Failed / null	14	25.0% of all

5.4 Study 3: Tactic Implementation on Architecture-Labeled Repositories52

TABLE 5.11
 ΔMI statistics for completed repositories (N = 42).

Metric	Value
Mean ΔMI (all completed)	+1.484
95% CI for $\overline{\Delta MI}$	[+0.26, +3.51]
Mean ΔMI (improved only)	+3.716
Mean ΔMI (worsened only)	−0.653

Compared to the initial pipeline experiment (Section 5.2), both the improvement rate (42.9% vs. 18.2%) and the mean gain (+1.484 vs. +0.48) are substantially higher. This is consistent with the hypothesis that validated architecture labels produce more coherent tactic selection.

Results by Tactic, Architecture, and Repository Size

Table 5.12 shows results broken down by the three primary grouping variables.

5.4 Study 3: Tactic Implementation on Architecture-Labeled Repositories

TABLE 5.12
Results by tactic, architectural style, and repository size.

Group	Category	N	Impr.	Worse.	$\overline{\Delta MI}$
Tactic	Decomposability	18	8	3	+2.33
	Localized Modification	14	9	2	+1.49
	Reduced Coupling	3	1	2	-0.16
	(none)	7	0	0	0.00
Architecture	Script-based	7	3	0	+5.62
	Modular monolith	26	9	5	+0.76
	Layered	9	6	2	+0.35
Size	Tiny (<10 files)	11	—	—	median $ \Delta = 1.04$
	Small (10–30 files)	9	—	—	median $ \Delta = 0.76$
	Medium (30–80 files)	12	—	—	median $ \Delta = 0.05$
	Large (>80 files)	10	—	—	median $ \Delta = 0.00$

Tactic. Localized Modification achieves the best risk-adjusted performance (9 improvements, 2 worsenings), while Decomposability is high-variance: it produces the largest gains (+21.98) but also generates more worsenings (3 cases) when new extracted modules have MI below the repository average.

Architectural style. Script-based repositories produce the highest mean improvement (+5.62) due to their few files and clear decomposition opportunities. Modular monoliths account for all five worsening cases, confirming that already-structured codebases are sensitive to structural interventions. Layered repositories show the highest improvement rate in count (6 of 9), suggesting that explicit layer boundaries provide a structured context for Localized Modification.

Repository size. Size is the dominant moderating variable. On tiny repositories, a single file split can raise the repository-level MI by over 20 points because

5.4 Study 3: Tactic Implementation on Architecture-Labeled Repositories⁵⁴

the average is computed over only a few files. On large repositories (80+ files), individual changes are diluted by averaging and the median $|\Delta MI|$ is effectively zero. The correlation between Python file count and $|\Delta MI|$ is -0.13 , confirming a weak but consistent inverse relationship.

Key Improvement Mechanism

Across 12 of the 18 improved repositories, the core mechanism is the extraction of a new module achieving $MI = 100$. These modules contain cohesive, simple code: configuration objects, pure utility functions, or API clients with limited branching. A secondary pattern involves files with $MI = 0$ (Radon rank C): four repositories improved by at least 7 MI points after the LLM extracted part of a critically complex file, because the Halstead Volume component of MI decreases non-linearly with file size. Representative examples are:

- *kodekloudhub/webapp-color* ($\Delta MI = +21.98$): a single `app.py` was split into `app.py`, `color.py` ($MI = 100$), and `config.py` ($MI = 100$).
- *AutoLab-SAI-SJTU/Paper2Rebuttal* ($\Delta MI = +18.14$): `rebuttal_service.py` ($MI = 0$) was decomposed into three focused modules; the original file recovered to $MI = 55.6$.
- *HKUDS/FastCode* ($\Delta MI = +2.11$, 81 files): a rare large-repository success; `retriever.py` improved from $MI = 11.3$ to 78.4 and `main.py` from 4.9 to 77.0 through targeted extraction.

Failure and Degradation Analysis

Stable outcomes (17 cases) arise from three causes: generated files containing syntax errors that Radon excludes from the average (3 cases), planner failures

in which the agent cannot produce a valid modification plan after repeated retries (7 cases), and no-op changes such as docstring additions that do not affect MI. Worsening outcomes (7 cases) occur when newly created files have MI below the existing repository average; all seven are bounded above -1.24 MI points and all occur in small or tiny repositories.

Technical pipeline failures include Ollama API rate limiting (HTTP 429), repeated planner JSON parse errors, stuck modification loops, and invalid step proposals. These failure modes indicate that multi-step agentic LLM pipelines require more robust planning validation and loop-termination strategies.

5.5 Architectural Analysis of Changes

The analysis in this section draws on the 42 repositories with complete before/after static analysis data from Study 3. The goal is to characterize whether the pipeline produces architecture-level changes or only code-level modifications, using structural proxy metrics beyond MI.

5.5.1 Architectural Metrics Overview

The static analysis artifacts provide three levels of architectural indicators:

- **Code-level (MI):** per-file Maintainability Index via Radon, aggregated as repository mean.
- **Structural proxies:** `architecture_proxies.json` records file count, package count (directories with `__init__.py`), max directory depth, and the full import dependency graph.

- **Architecture maintainability:** `architecture_maintainability.json` records package count, average fan-out, max directory depth, and qualitative factor labels.

5.5.2 Package Structure and Directory Depth

Across all 42 completed repositories, **package count did not change in a single case**. The `packages` field in `architecture_maintainability.json` remained identical before and after in every repository. Maximum directory depth changed in only 2 of 42 cases (HerokuFreeProxy: $2 \rightarrow 3$, autoforge: $3 \rightarrow 4$). This confirms that the pipeline operates entirely within the existing module hierarchy and does not introduce new package-level organization.

5.5.3 Fan-Out Analysis

Average fan-out ($\overline{fan_out}$) improved in 15 of 42 repositories. Table 5.13 shows the four repositories with the largest fan-out reductions.

TABLE 5.13
Repositories with the largest average fan-out reductions.

Repository	Before	After	Δ	Nature
Paper2Rebuttal	12.6	10.5	−2.10	Genuine decomposition
webapp-color	5.0	3.33	−1.67	Mechanical file split
spam-api-free-fire	5.0	3.6	−1.40	Mechanical file split
fastapi-whisper-ollama	2.0	1.56	−0.44	Strategy pattern extraction

Fan-out improves when new files are created and imports are redistributed. Repositories with established layered or modular monolith structures show negli-

gible fan-out changes, because their inter-module dependency patterns are already stable.

5.5.4 MI vs. Architectural Value: A Critical Comparison

Table 5.14 compares two repositories with similar ΔMI but fundamentally different architectural outcomes.

TABLE 5.14
Comparison of mechanical vs. genuine architectural improvement.

Metric	webapp-color	Paper2Rebuttal
ΔMI	+21.98	+18.14
Δ files	+2	+3
Δ fan-out	−1.67	−2.10
Δ packages	0	0
Nature	Mechanical split	Genuine decomposition

webapp-color (script-based) splits a single `app.py` into `app.py` + `color.py` + `config.py`. The two extracted files each have $MI = 100$ because they contain trivial code (configuration values, simple color conversion). The +21.98 MI improvement is an arithmetic artifact of averaging: MI increases because the Halstead Volume is distributed across multiple files, not because the architecture improved. This **confirms Devil’s Advocate criticism C2**: mechanical file splitting trivially improves MI .

Paper2Rebuttal (modular monolith) extracts three focused modules from large mixed-responsibility files: `pdf_converter.py` from `tools.py` (500+ lines), `arxiv_client.py` from `arxiv.py` (300+ lines), and `llm_orchestrator.py` from `rebuttal_service.py` (500+ lines, $MI = 0$). This represents genuine architectural

improvement: separation of concerns, reduced coupling, and isolation of cross-cutting logic. Yet the MI improvement (+18.14) is comparable to the mechanical webapp-color case.

MI does not distinguish between these two cases. The metric captures file-level complexity reduction regardless of whether the reduction reflects meaningful decomposition or trivial splitting. This is the core construct validity challenge (DA C1): MI is a necessary but insufficient measure for evaluating architecture-level changes.

5.5.5 Correlation Analysis

The Spearman correlation between Δ files and ΔMI across all 42 repositories is $\rho = 0.74$ ($p < 0.001$). The correlation between Δ fan-out and ΔMI is $\rho = -0.11$ (not significant). The number of new files created is the strongest structural predictor of MI improvement, consistent with the mechanical-splitting explanation. Package count shows zero variance and cannot be correlated.

5.5.6 Summary of Architectural Findings

1. **No architectural restructuring occurred:** package count unchanged in 42/42 cases, directory depth changed in 2/42.
2. **MI does not distinguish architectural from non-architectural changes:** webapp-color (mechanical split) and Paper2Rebuttal (genuine decomposition) produce nearly identical MI gains.
3. **The primary driver of ΔMI is file splitting:** correlation $\rho = 0.74$ between Δ files and ΔMI ; the sensitivity analysis confirms that removing three

extreme splitting cases eliminates statistical significance.

4. **Reduced Coupling is ineffective:** none of the 6 repositories assigned this tactic showed meaningful improvements; the tactic is the worst-performing.
5. **The pipeline operates at the code level:** all changes are file-level extractions and local refactorings within the existing module hierarchy.

These findings support a reframing of the thesis contribution: the pipeline implements **code-level tactic implementation guided by architectural context**, not architecture-level restructuring. The contributions are in understanding the capabilities and limitations of LLMs for code-level tactic application, the effectiveness of architecture detection, and the gap between code-level changes and architectural transformation.

5.6 Cost-Benefit Estimate

To assess practical viability, execution logs from Study 3 were analyzed for API consumption. Across 56 repositories, the pipeline made an average of 12.4 LLM calls per repository (range: 3–47), consuming approximately 185,000 tokens per repository (input) and 8,400 tokens (output). At current Ollama cloud API pricing, the average cost per repository is approximately \$0.32 for small repositories (under 30 files) and \$1.15 for large repositories (over 80 files). Execution time averaged 6.2 minutes per repository, dominated by LLM inference latency.

At a mean ΔMI of +1.484 for validated labels, the cost per MI point gained is approximately \$0.22 for small repositories. However, the mean hides large

variance: 40.5% of completed repositories showed no MI change, effectively making their cost \$0.32–\$1.15 for zero improvement. For practitioners, the break-even question depends on the value assigned to a 1-point MI improvement over the project lifetime. For a small script-based repository where a +5–20 point gain is achievable, the investment is trivially small relative to the benefit. For a modular monolith where the expected gain is near zero, the investment is not justified.

5.7 Automation Bias and Training-Data Bias

The systematic misclassification of modular monolith as layered across all five models (Section 5.3.4) is consistent with an automation bias hypothesis. LLM training data is dominated by layered MVC architectures from web frameworks (Django, Flask, FastAPI tutorials and examples). When models encounter a modular monolith organized by domain features, they may default to the overrepresented layered template. This directional bias does not diminish with richer input ($P2 \rightarrow P3 \rightarrow P4$), suggesting it is a learned prior rather than an information-deficit problem.

This bias has practical implications: any downstream system that consumes LLM-based architecture labels for tactic selection will systematically over-apply layered-appropriate tactics to domain-organized repositories. In the pipeline, this means modular monoliths may receive coupling-reduction tactics designed for technical layer hierarchies rather than the domain-boundary tactics they require.

Future work should investigate debiasing strategies: counterexample prompting, architectural conformance rule injection, or hybrid approaches that combine LLM classification with static structural thresholds.

5.8 Integrated Discussion

The three studies and the architectural analysis together provide a multi-level view of what works and what is currently limited in LLM-based code-level tactic implementation guided by architectural context.

5.8.1 Architecture Detection Is Feasible but Boundary-Limited

Study 2 shows that LLM-based architecture detection can produce above-baseline accuracy with a lightweight scriptable pipeline. However, 70.2% accuracy means approximately 30% of repositories are misclassified, predominantly modular monoliths classified as layered. This directional bias is consistent across all five models and does not improve with richer input, suggesting it reflects a learned prior from training data dominated by layered MVC architectures.

For the pipeline, this error is particularly costly: a domain-organized modular monolith may receive tactic recommendations designed for a technical layer hierarchy. The degradation cases in Study 1 (all seven occurring in modular monolith repositories) are consistent with this pathway.

5.8.2 Import Graphs Are the Most Valuable Structural Signal

Import dependency graphs consistently improve classification across all five models. Code signatures consistently degrade or do not change performance. The design recommendation for any LLM-based architecture analysis system is unambiguous: include import dependency graphs, omit code signatures.

5.8.3 LLM Transformations Remain Code-Level Rather Than Architecture-Level

The architectural analysis (Section 5.5) provides the clearest evidence: package count remained unchanged in 42/42 repositories, directory depth changed in 2/42, and Δ files correlates with ΔMI at $\rho = 0.74$. The pipeline operates at the code level — file splitting and local extraction within the existing module hierarchy — not at the architectural level of module boundary restructuring or package reorganization.

This reframing is the most important finding of the thesis. The original framing of “architecture-aware maintainability improvement” sets an expectation the evidence does not support. The more accurate framing is **code-level tactic implementation guided by architectural context**. The gap between tactic intent (architectural) and implementation scope (code-level) is the central limitation, consistent with the broader observation in LLM-based refactoring research that agentic refactorings are dominated by localized edits [22], [26].

5.8.4 MI Does Not Measure Architectural Improvement

The comparison of webapp-color (mechanical file split, $\Delta MI = +21.98$) and Paper2Rebuttal (genuine decomposition, $\Delta MI = +18.14$) demonstrates that MI cannot distinguish architecturally meaningful changes from trivial restructuring. The sensitivity analysis confirms that removing the top three extreme improvements eliminates statistical significance. MI is a valid measure of file-level complexity change but is not a proxy for architectural maintainability.

This validates the Devil’s Advocate critique (C1, C2) and motivates the thesis’s reframing. Future work should supplement MI with architecture-level met-

rics such as cyclic dependency ratio, inter-module coupling intensity, or package tangle percentage, which can be computed from the existing import dependency graph data collected in this study.

5.8.5 Script-Based Repositories Are the Strongest Use Case

The strongest improvements occur in script-based repositories. This represents a practical sweet spot: low-structure repositories offer clear decomposition targets, and even simple modularization produces measurable metric gains. For small script-based repositories with files scoring MI near zero, the pipeline produces reliable improvements at low cost (approximately \$0.32 per repository).

5.8.6 Reduced Coupling Is Ineffective

Reduced Coupling is the worst-performing tactic across both Study 1 and Study 3. It is the only tactic with a negative mean ΔMI , and none of 6 assigned repositories in Study 3 showed meaningful improvement. LLMs struggle to reduce inter-module coupling because the tactic requires coordinated changes across multiple files with global architectural understanding — precisely the capability limitation identified in recent literature [22].

5.8.7 The Confidence Problem Propagates Across Studies

Self-reported LLM confidence does not distinguish correct from incorrect predictions (maximum $\Delta = 0.026$; 95% CI $[-0.01, +0.04]$). Study 1 relied on these confidence values, meaning tactic selection may have been partially based on incorrect classifications. Future iterations should not use confidence as a

reliability indicator.

5.8.8 Failure Rate Requires Verification Mechanisms

The 25.3% failure rate indicates that the current implementation is not reliable enough for autonomous deployment. The pipeline provides syntactic validation but not behavioral preservation. Production deployment would require automated test suite execution, build validation, rollback, and human review for high-risk transformations.

5.9 Threats to Validity

5.9.1 Internal Validity

In Study 1, each repository serves as its own control, reducing cross-project variance. However, MI changes may be influenced by factors beyond the intended tactic, such as code formatting side effects or coincidental complexity changes during modification. A single LLM model was used without comparison to alternatives or manual implementation of tactics. The sensitivity analysis (Section 5.2.2) shows that the significant result is driven by a small number of extreme improvements, suggesting that the aggregate positive effect is not uniformly distributed across the dataset.

In Study 2, the ground truth labels were assigned by a single annotator. The modular monolith/layered boundary is inherently ambiguous, and a second annotator may draw the boundary differently for some repositories. Inter-annotator reliability (Cohen's κ) was not measured, which is a recognized limitation given that previous work has shown low agreement in architectural style classification.

Each prompt was executed once per model; the low temperature setting (0.2) reduces but does not eliminate stochastic variance.

The multiple-comparisons issue in Study 2 (20 model-prompt pairs) is bounded by the Bonferroni-Holm correction discussed in Section 3.9, but the primary findings (import graphs help all models, code signatures help none) are replicated across five independent models, making them robust to this concern.

5.9.2 External Validity

All three studies are limited to Python backend repositories from GitHub. The taxonomy covers three architectural styles; microservice, event-driven, and hexagonal architectures are not represented. Results may not generalize to other languages, enterprise systems, or more architecturally complex repositories.

5.9.3 Construct Validity

The Maintainability Index is a composite metric that does not capture all dimensions of architectural maintainability. The architectural analysis (Section 5.5) demonstrates that MI cannot distinguish between mechanically split files and genuine architectural decomposition. This is the most significant construct validity threat in the thesis. MI is sensitive to file-level complexity (Halstead Volume, Cyclomatic Complexity) but insensitive to modularity, architectural conformance, and inter-module dependency quality. The thesis reframes its contributions as “code-level tactic implementation guided by architectural context” to align with what MI actually measures.

Architecture style is a high-level abstraction that some repositories may exhibit in hybrid form. Forcing repositories into a single class label may not

reflect their actual structure.

5.9.4 Automation Bias

LLMs encode specific code patterns from their training data. The systematic misclassification of modular monolith as layered across all five models (Section 5.3.4) is consistent with a learned preference for layered MVC architectures, which dominate the training distribution. This directional bias means that architecture detection may systematically mischaracterize well-structured non-layered systems, potentially biasing downstream tactic selection toward layer-appropriate tactics (e.g., Layered Separation) over domain-appropriate ones (e.g., Localized Modification along domain boundaries).

5.9.5 Conclusion Validity

The datasets used in all three studies are relatively small. Study 2 uses 57 repositories; Study 1 uses 162 repositories with 121 completed. Statistical power is limited, and all results should be interpreted as early empirical evidence rather than definitive claims. All rankings and effect sizes are preliminary baselines.

Chapter 6

Conclusion

This thesis investigated whether Large Language Models can support software maintainability improvement through code-level tactic implementation guided by architectural context. The investigation consisted of three empirical studies and an architectural analysis covering the full workflow from architecture detection to tactic implementation and maintainability measurement. This chapter summarizes the findings with respect to the research objectives, identifies the main limitations, and proposes directions for future research.

6.1 Summary of Contributions

The thesis makes three primary contributions.

An empirical evaluation of LLM-based architecture detection. The study provides the first systematic multi-model evaluation of repository-level architectural style classification using LLMs. Five models were evaluated across four context configurations on a manually labeled dataset of 57 Python backend repositories. The evaluation yields clear design recommendations: import dependency

graphs are the most valuable evidence type, code signatures should be omitted, and LLM confidence scores are insufficiently calibrated to be used as reliability indicators. A multiple-comparisons analysis bounds the statistical significance of individual model comparisons.

An empirical evaluation of LLM-implemented architectural tactics with critical construct validity analysis. The thesis evaluates an LLM-based tactic implementation pipeline across 162 repositories and subsequently with validated architecture labels across 57 repositories. The results quantify the relationship between architecture label quality and implementation outcomes, and identify repository size as the dominant moderating factor. The architectural analysis (Section 5.5) demonstrates that MI does not distinguish genuine architectural restructuring from mechanical file splitting — the most significant construct validity finding — and that all pipeline changes operate at the code level rather than the architectural level. The thesis reframes its central claim accordingly.

A reusable pipeline architecture and artifact dataset. The pipeline implementation, labeled dataset, extracted repository artifacts, prompt templates, and raw model outputs are made publicly available, enabling replication and extension of the reported experiments.

6.2 Findings with Respect to Research Objectives

Objective 1: Identify architectural tactics that influence maintainability. The thesis grounds tactic selection in four tactics applicable to Python backend repositories: Decomposability, Localized Modification, Reduced Coupling, and Deferred Binding Time. Tactics requiring runtime infrastructure, organizational processes, or deployment-level restructuring are deliberately excluded. Empirical

results confirm that Decomposability and Localized Modification produce measurable MI improvements in receptive repositories, while Reduced Coupling is consistently ineffective — the only tactic with a negative mean ΔMI across both studies.

Objective 2: Implement selected tactics using LLMs. The pipeline successfully selects and implements tactics in 74.7% of attempted repositories through an agentic loop with BM25-based file retrieval, planner-based step selection, patch generation, and syntactic validation. The implementation reliably produces code-level changes (file splits, extractions) but does not produce architecture-level changes (no package count changes, negligible directory depth or fan-out improvements).

Objective 3: Evaluate the impact using metrics. MI is sensitive to file-level complexity changes but does not distinguish architectural improvement from mechanical file splitting (webapp-color vs. Paper2Rebuttal: ΔMI of +21.98 vs. +18.14 with fundamentally different architectural value). Package count remained unchanged across all repositories. Fan-out changed significantly in only 4 of 42 repositories. The sensitivity analysis confirms that the statistical significance is driven by a small number of extreme file-splitting cases.

Objective 4: Compare maintainability before and after transformations. The pipeline produces a statistically significant but fragile MI shift ($W = 360.0$, $p = 0.001$, $r = 0.28$; mean +0.48, 95% CI [+0.18, +0.82]) on 162 repositories. Significance is lost when the top three extreme improvements are excluded. Using validated architecture labels increases the improvement rate from 18.2% to 42.9% and the mean gain from +0.48 to +1.484, confirming that detection quality propagates into implementation outcomes.

Objective 5: Evaluate advantages and limitations of LLMs for code-level

tactic implementation.

On the advantage side, LLMs can reason about repository-level structure, select contextually appropriate tactics, and generate code modifications that produce measurable (if localized) maintainability improvements without predefined transformation rules.

On the limitation side, five issues are consistently observed. First, architecture detection errors — particularly the systematic confusion of modular monolith with layered — propagate into tactic selection. Second, the implementation operates at the code level, producing file splits rather than module boundary restructuring. Third, the 25.3% failure rate reflects brittleness in multi-step LLM code generation. Fourth, LLM confidence scores are uncalibrated. Fifth, MI as the primary dependent variable cannot distinguish architectural from non-architectural changes, requiring a reframing of the thesis contribution.

Objective 6: Formulate recommendations. The empirical results support the following recommendations:

- Use file tree combined with import dependency graph as the default evidence configuration for architecture detection. Code signatures should be omitted.
- Do not rely on LLM confidence scores to identify unreliable architecture predictions. Human review is necessary where the modular monolith/layered boundary is relevant downstream.
- Prioritize small, script-based repositories with files scoring near-zero MI as primary targets. At approximately \$0.32 per repository, the investment is justified for potential +5–20 point gains.

- Apply Localized Modification in preference to Decomposability when degradation risk must be minimized. Avoid Reduced Coupling — it is consistently ineffective.
- Integrate test-based regression detection in any production deployment. Supplement MI with architecture-level metrics (cyclic dependency ratio, coupling intensity) for meaningful evaluation of structural changes.
- Frame LLM-based tactic implementation as *code-level improvement guided by architectural context*, not as autonomous architectural refactoring. The gap between code-level execution and architecture-level restructuring is the central limitation.

6.3 Limitations

The conclusions of this thesis are bounded by several limitations that should be considered when interpreting the results.

Single annotator. Architecture ground truth labels were assigned by one annotator. The modular monolith/layered boundary is the hardest annotation decision, and inter-annotator agreement (Cohen’s κ) was not measured. Some repositories may genuinely exhibit hybrid characteristics. This limitation is recognized by all five reviewers as a critical threat requiring a second annotator before publication-quality claims can be made.

Dataset scope. All three studies are restricted to Python backend repositories from GitHub with `requirements.txt` as a hard inclusion criterion. This excludes modern Python projects using `poetry`, `pipenv`, or `pyproject.toml` alone, potentially biasing the dataset toward older or less modern projects. The taxon-

omy covers three architectural styles; microservice, event-driven, and hexagonal architectures are not represented. Results may not generalize to other languages, enterprise systems, or non-backend domains.

MI construct validity (CRITICAL). The Maintainability Index is sensitive to file-level complexity but insensitive to module boundary quality, architectural conformance, and inter-module coupling changes. The architectural analysis demonstrates that MI cannot distinguish mechanical file splitting from genuine architectural decomposition. This is the most significant validity threat, recognized as CRITICAL by the Devil’s Advocate review and acknowledged by all other reviewers. The thesis reframes its contributions to align with what MI actually measures.

No baseline comparison. The pipeline is never compared against alternatives: random file splitting, static-analysis-only refactoring, or manual implementation by a developer. Without baselines, the results demonstrate absolute capability but not relative effectiveness. This is a major limitation that prevents stronger causal claims about LLM-specific benefits.

Absence of behavioral verification. The pipeline validates syntactic correctness but does not guarantee semantic behavior preservation. MI improvements may coexist with undetected behavioral regressions.

Single LLM for implementation. The implementation pipeline uses Qwen3-coder-next exclusively. The claim “LLMs can support maintainability improvement” conflates with “this specific LLM in this specific configuration,” though this limitation is prominently noted.

Preliminary nature. All three studies produce early feasibility evidence. Effect sizes are small to moderate, datasets are limited in size and diversity, and statistical power is constrained. The sensitivity analysis shows that statistical

significance in Study 1 is driven by a small number of extreme values.

Automation bias. LLMs encode training-data preferences that may systematically favor layered MVC architectures. The systematic modular-monolith-to-layered misclassification across all five models is consistent with this bias, which may affect the generalizability of architecture detection results to non-web-dominated codebases.

6.4 Future Work

The findings of this thesis point to several directions for future investigation.

Multi-annotator ground truth. Expanding the architecture detection dataset with multiple annotators and measuring inter-rater agreement (Cohen’s κ) would establish a more reliable benchmark, particularly for the modular monolith/layered boundary.

Baseline comparison. Adding comparison conditions — random file splitting, static-analysis-only tools, or manual developer implementation — would transform the evaluation from absolute capability assessment to relative effectiveness comparison.

Behavioral preservation verification. Integrating automated test suite execution as a mandatory pipeline stage would transform the pipeline from an experimental mechanism into a behaviorally safe transformation tool.

Architecture-level metrics. Computing cyclic dependency ratio, inter-module coupling intensity, and package tangle percentage from the existing import dependency graph data would provide more discriminating evaluation of architectural tactic effects.

Broader taxonomy and languages. Extending to microservice, event-

driven, and hexagonal styles, and evaluating on Java, TypeScript, or Go repositories, would establish generalizability.

Cost-benefit with richer baselines. Estimating the person-minutes saved per MI point gained, and comparing against human refactoring effort, would ground the practical contribution.

Debiasing architecture detection. Counterexample prompting, architectural conformance rule injection, or hybrid LLM+static approaches could reduce the systematic layered bias in modular monolith detection.

Adaptive context selection. Varying the evidence configuration based on repository size or detected structural characteristics could improve both classification accuracy and implementation coherence.

Multi-model implementation comparison. Evaluating multiple LLMs on the full implementation pipeline would clarify whether the code-level limitation is model-specific or reflects a general capability gap.

Bibliography cited

- [1] A.-J. Molnar and S. Motogna, “A study of maintainability in evolving open-source software,” in *International Conference on Evaluation of Novel Approaches to Software Engineering*, Springer, 2020, pp. 261–282.
- [2] *ISO/IEC 25010 — Systems and software engineering: Software product Quality Requirements and Evaluation (SQuaRE) — Product quality model*, <https://iso25000.com/en/iso-25000-standards/iso-25010>, [Online; accessed 2025-12-04], 2023.
- [3] S. Kim, D.-K. Kim, L. Lu, and S. Park, “Quality-driven architecture development using architectural tactics,” *Journal of Systems and Software*, vol. 82, no. 8, pp. 1211–1231, 2009.
- [4] M. Kassab, M. Mazzara, J. Lee, and G. Succi, “Software architectural patterns in practice: An empirical study,” *Innovations in Systems and Software Engineering*, vol. 14, no. 4, pp. 263–271, 2018. DOI: [10.1007/s11334-018-0319-4](https://doi.org/10.1007/s11334-018-0319-4)
- [5] D. E. Perry and A. L. Wolf, “Foundations for the study of software architecture,” *ACM SIGSOFT Software Engineering Notes*, vol. 17, no. 4, pp. 40–52, 1992. DOI: [10.1145/141874.141884](https://doi.org/10.1145/141874.141884)

- [6] K. DePalma, I. Miminoshvili, C. Henselder, K. Moss, and E. A. AlOmar, “Exploring ChatGPT’s code refactoring capabilities: An empirical study,” *Expert Systems with Applications*, vol. 249, p. 123 602, 2024.
- [7] D. Garlan and M. Shaw, “An introduction to software architecture,” *Advances in software engineering and knowledge engineering*, vol. 1, no. 3, 1993.
- [8] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th. Addison-Wesley, 2021, ISBN: 978-0136886099.
- [9] M. Kim, T. Zimmermann, and N. Nagappan, “An empirical study of refactoring challenges and benefits at Microsoft,” *IEEE Transactions on Software Engineering*, vol. 40, no. 7, pp. 633–649, 2014. DOI: [10.1109/TSE.2014.2318734](https://doi.org/10.1109/TSE.2014.2318734)
- [10] M. H. Halstead, *Elements of Software Science*. Elsevier, 1977.
- [11] T. J. McCabe, “A complexity measure,” *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976. DOI: [10.1109/TSE.1976.233837](https://doi.org/10.1109/TSE.1976.233837)
- [12] P. Oman and J. Hagemeister, “Metrics for assessing a software system’s maintainability,” *Conference on Software Maintenance*, pp. 337–344, 1992.
- [13] V. Lenarduzzi, F. Pecorelli, N. Saarimaki, S. Lujan, and F. Palomba, “A critical comparison on six static analysis tools: Detection, agreement, and precision,” *Journal of Systems and Software*, vol. 198, p. 111 575, 2023.
- [14] N. B. Harrison and P. Avgeriou, “How do architecture patterns and tactics interact? A model and annotation,” *Journal of Systems and Software*, vol. 83, no. 10, pp. 1735–1758, 2010.

- [15] R. Li, P. Liang, M. Soliman, and P. Avgeriou, “Understanding architecture erosion: The practitioners’ perceptive,” in *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*, IEEE, 2021, pp. 311–322.
- [16] J. Rosik, A. Le Gear, J. Buckley, M. A. Babar, and D. Connolly, “Assessing architectural drift in commercial software development: A case study,” in *Software: Practice and Experience*, vol. 41, Wiley Online Library, 2011, pp. 63–86.
- [17] T. Bi, P. Liang, A. Tang, and X. Xia, “Mining architecture tactics and quality attributes knowledge in Stack Overflow,” *Journal of Systems and Software*, vol. 180, p. 111 005, 2021. DOI: [10.1016/j.jss.2021.111005](https://doi.org/10.1016/j.jss.2021.111005)
- [18] G. Márquez, H. Astudillo, and R. Kazman, “Architectural tactics in software architecture: A systematic mapping study,” *Journal of Systems and Software*, vol. 197, p. 111 558, 2022. DOI: [10.1016/j.jss.2022.111558](https://doi.org/10.1016/j.jss.2022.111558)
- [19] J. Bogner, S. Wagner, and A. Zimmermann, “Using architectural modifiability tactics to examine evolution qualities of service- and microservice-based systems,” *Computer Science – Research and Development*, vol. 34, pp. 187–237, 2019. DOI: [10.1007/s00450-019-00402-z](https://doi.org/10.1007/s00450-019-00402-z)
- [20] Z. Rahmati and M. Tanhaei, “Ensuring software maintainability at software architecture level using architectural patterns,” *AUT Journal of Mathematics and Computing*, vol. 2, no. 1, pp. 81–102, 2021. DOI: [10.22060/ajmc.2021.19232.1044](https://doi.org/10.22060/ajmc.2021.19232.1044)
- [21] Y. Xu, F. Lin, J. Yang, T.-H. Chen, and N. Tsantalis, “MANTRA: Enhancing automated method-level refactoring with contextual RAG and multi-agent

- LLM collaboration,” in *arXiv preprint arXiv:2503.14340*, 2025. arXiv: [2503.14340](#).
- [22] K. Horikawa, H. Li, Y. Kashiwa, B. Adams, H. Iida, and A. E. Hassan, “Agentic refactoring: An empirical study of AI coding agents,” in *arXiv preprint arXiv:2511.04824*, 2025. arXiv: [2511.04824](#).
- [23] B. Liu, Y. Jiang, Y. Zhang, N. Niu, G. Li, and H. Liu, “Exploring the potential of general purpose LLMs in automated software refactoring: An empirical study,” *Automated Software Engineering*, vol. 32, no. 1, 2025. DOI: [10.1007/s10515-025-00500-0](#)
- [24] Y. C. K. Piao, J. C. Paul, L. Da Silva, A. M. Dakhel, M. Hamdaqa, and F. Khomh, “Refactoring with LLMs: Bridging human expertise and machine understanding,” in *arXiv preprint arXiv:2510.03914*, 2025. arXiv: [2510.03914](#).
- [25] S. E. Shokri and R. Khatchadourian, “IPSynth: Interprocedural program synthesis for software architecture recovery and architectural tactics detection,” in *arXiv preprint arXiv:2403.10836*, 2024. arXiv: [2403.10836](#).
- [26] S. Martinez, L. Xu, M. Elnaggar, and E. A. AlOmar, “Software refactoring research with large language models: A systematic literature review,” in *Journal of Systems and Software*, Elsevier, 2025, p. 112 762. DOI: [10.1016/j.jss.2025.112762](#)
- [27] A. P. Pucho, A. M. Ferreira, E. J. R. Cirilo, and B. B. Cafeo, “Refactoring python code with llm-based multi-agent systems: An empirical study in ml software projects,” in *Simpósio Brasileiro de Engenharia de Software (SBES)*, SBC, 2025, pp. 678–684.

-
- [28] M. Esposito et al., “Generative ai for software architecture: Applications, trends, challenges, and future directions,” *Applications, Trends, Challenges, and Future Directions*, 2025.
 - [29] S. Yao et al., “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
 - [30] F. Zhang et al., “Repocoder: Repository-level code completion through iterative retrieval and generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2471–2484.
 - [31] S. Robertson and H. Zaragoza, *The probabilistic relevance framework: BM25 and beyond*. Now Publishers Inc, 2009, vol. 4.

Appendix A

Implementation Parameters

This appendix documents the detailed implementation parameters of the tactic implementation subsystem, which were condensed from the main text for readability.

A.1 BM25 Retrieval

File retrieval uses the BM25 ranking algorithm with $k_1 = 1.5$ and $b = 0.75$, following standard information retrieval defaults [31]. The query is constructed from the tactic name, category, and description. Each file's document is formed by concatenating its path, class names, function names, and import list. The top $k = 5$ files with positive BM25 scores are passed to the Planner Agent.

A.2 Iteration Control

The agentic loop terminates when any of the following conditions is met:

- Maximum iterations reached (default: 5);

- Planner Agent issues a STOP signal;
- Planner exceeds 3 consecutive parse or validation failures.

A.3 Patch Generation Constraints

The Patch Agent returns the full replacement file content (not a diff). Generated files are rejected if they exceed 400 lines or target `__init__.py`. The agent is instructed to make one focused change per step and to preserve existing external interfaces.

A.4 Self-Healing

When a regression is detected, the affected file is restored from the pre-patch backup and the Patch Agent is re-invoked with test failure output as feedback. Up to 2 repair attempts are permitted per iteration. If all attempts fail, the iteration is abandoned and the pipeline continues to the next iteration.

A.5 Backup Directory Structure

File-level backups are organized per iteration and attempt:

`backups/<repository>/step_<N>/attempt_<M>/<file_path>`

This structure enables precise rollback of individual files without reverting unrelated changes from other iterations.

A.6 Artifact Storage

For each repository, the following artifacts are preserved:

- **Structural index:** JSON file with per-file path, imports, class names, function names, and size.
- **Test baseline:** pre-modification pytest output with failure list.
- **Step artifacts:** one JSON patch per completed iteration.
- **Test results:** pytest output, failure names, regression flag, and success status per iteration.
- **Artifact extraction:** file tree, import graph (text and JSON), code signatures.

A.7 Pipeline Execution Workflow

Per repository, the system executes sequentially:

1. Shallow clone repository;
2. Extract file tree, import graph, code signatures, structural metrics;
3. Compute baseline MI, fan-out, package count, docstring coverage;
4. Run LLM-based architecture detection;
5. Run LLM-based tactic selection;
6. Apply LLM-generated modifications via agentic loop;

7. Validate syntactic correctness of modified files;
8. Recompute all metrics on modified repository;
9. Store results and logs;
10. Delete local repository clone.